Lecture 3

10 October 2026 20:16

Making of the generation function

This will help to generate the next token

----------THE WHOLE CODE----------

```
def generate(self, idx, max_new_tokens):
    # idx is a (B, T) tensor of token IDs in the current context
    for _ in range(max_new_tokens):
        # STEP 1: Forward pass to get predictions
        logits, loss = self(idx) # logits shape: (B, T, C)

        # STEP 2: Pluck out only the very last character's logits
        logits = logits[:, -1, :] # Shape changes from (B, T, C) to (B, C)

        # STEP 3: Convert logits to probabilities using Softmax
        probs = F.softmax(logits, dim=-1) # Shape: (B, C)

        # STEP 4: Sample 1 next token from the probability distribution
        idx_next = torch.multinomial(probs, num_samples=1) # Shape: (B, 1)

        # STEP 5: Append the newly generated token to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # Shape becomes (B, T+1)

    return idx
```

Remember : till now we have written only the forward pass logic, which only tell how the input will move in a neural network

1. The Starting Context (idx)
When we start generating from scratch, we often pass a single newline character \n (ID = 0) as a starting seed:

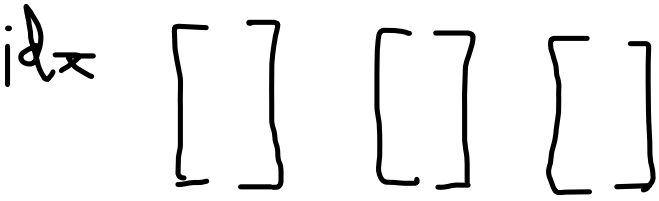
python
```
idx = torch.zeros((1, 1), dtype=torch.long) # Shape: (B=1, T=1)
```

2. STEP 1: logits, loss = self(idx)

We feed idx into the model (calling forward()):

1. Because we did NOT pass targets, loss is None.
2. logits has shape (B, T, C).
3. If our input text has [Equation] letters so far, logits will have shape (1, 5, 65).

So the index(idx) will be feed in the layers of the neural network



The reason why we have first logit s (1, 5, 65) and not (4, 5, 65) cause we are accessing first row of the input batch
So, it means that the input is from 1 cross 5 (word size) and refer through 65 vocab size

3. STEP 2: Why logits[:, -1, :]?

In Python, index -1 means **the very last item**.

- logits[:, -1, :] means:
  - Take all batches (:) ----------> taking all the 4 rows of the a single single batch
  - Take **only the last time step** (-1)
  - Take all 65 character scores (:)

- We only care about what comes **next** after the very last letter in the sequence.
- Shape shrinks from (B, T, C) to **(B, C)** (e.g., (1, 65)).

4. STEP 3: probs = F.softmax(logits, dim=-1)

Softmax takes those 65 raw scores and converts them into **probabilities** ([Equation] to [Equation]) that sum up to [Equation] (or [Equation]):

- Example:
  - 'e': [Equation] ([Equation] chance)
  - 'a': [Equation] ([Equation] chance)
  - 'z': [Equation] ([Equation] chance)

5. STEP 4: What is torch.multinomial(probs, num_samples=1)?

Why don't we just pick the character with the highest probability (torch.argmax)?

1. If you always pick the highest score, the model becomes completely deterministic, repetitive, and boring.
2. **torch.multinomial rolls a weighted die!**
   - If 'e' has a [Equation] chance, it will be chosen [Equation] of the time.
   - If 'a' has a [Equation] chance, it will occasionally be picked [Equation] of the time.
3. It returns idx_next with shape **(B, 1)** (e.g., tensor [[5]] for 'e').

6. STEP 5: torch.cat((idx, idx_next), dim=1)

torch.cat glues tensors together.

1. dim=1 is the sequence dimension ([Equation]).
2. V IMP :

If idx was "hell" ([Equation]) and idx_next is 'o' ([Equation] token), they get concatenated into "hello" ([Equation]).

3. Shape expands from (B, T) [Equation] **(B, T+1)**.

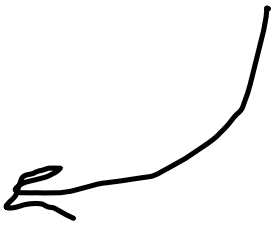
(B, T) means, it's the tensor of size(4, 8) which is nothing but the token's ID's of each word giving as input

Example: input is hello --- 'h' 'e' 'l' 'l' 'o'

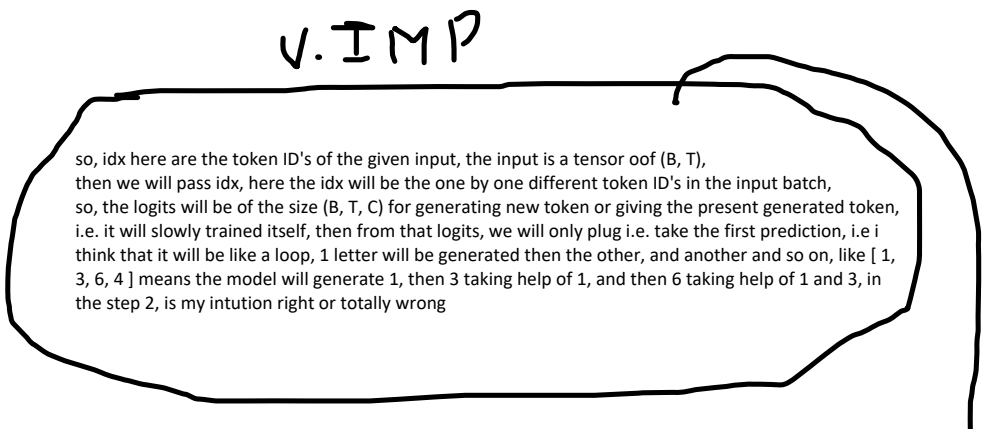
Step 1: (B, T) will be ---> batch 1 Seq 1 [h = 8] [e = 5] [l = 11] and so on
Seq 2
Seq 3
Seq 4

Step 2: ----------V IMP----------
When it is (B, T, C) it means, now it gives the outcomes (later probabilities of each alphabet coming after the particular single input token letter)

(B, T, C) will be ---> batch 1 Seq 1 [h = 8, 'a' = -2.45, 'b' = -2.16, ... upto 'z' = 1.80] [e = 5, 'a' = 1.20, 'b' = 6.20, ...and so on] [l = 11, similarly here also] and so on Seq 2
Seq 3
Seq 4



Batch 1:
Seq 1:
[-2.45, -2.16, ..., 1.80] <-- These 65 numbers replace 'h' (Predictions for what comes AFTER 'h')
[1.20, 6.20, ..., 0.05] <-- These 65 numbers replace 'e' (Predictions for what comes AFTER 'e')
[-0.55, 3.14, ..., -1.2] <-- These 65 numbers replace 'l' (Predictions for what comes AFTER 'l')
... (up to 8 vectors)



The only thing is the prediction from the every previous token(which have been already generated) is not done right now, later by self-attention mechanism, we do that

Explantation: Atleast read once for the video

Your Intuition (How full GPT / Transformers work)

In a real GPT model, if the sequence is [1, 3, 6]:

- To predict the next token, the model looks at **1, 3, AND 6** together.
- The model builds context: *"Oh, 1 was a noun, 3 was a verb, 6 is an adjective, so the next token should be a noun."*
- It uses all past history to make a smart prediction.

The Bigram "Gotcha" (How this specific model works)

The word **Bigram** means "two words/characters". This model is extremely simple and has **severe amnesia**.

- If the sequence is [1, 3, 6], the Bigram model completely ignores 1 and 3.
- It **only looks at 6** to predict the next token!
- Why? Because in the forward() pass:

  python
  logits = self.token_embedding_table(idx)
  The embedding table just does a 1-to-1 lookup. The vector for 6 has no idea that 1 and 3 came before it.

This is exactly why we do logits[:, -1, :] in Step 2!
Because the Bigram model doesn't allow tokens to "talk" to each other, passing the whole sequence [1, 3, 6] generates 3 separate, isolated predictions. We only care about what comes after the *very last* token (6), so we throw away the predictions for 1 and 3 by slicing out the very last item in the sequence: [:, -1, :].

Why does Karpathy write the code this way if the model is so dumb?

He is setting you up for the next part of the video!
Right now, the Bigram model generates terrible, random-looking text because it only looks 1 character in the past.
Soon, he will introduce **Self-Attention**. Self-Attention is the magic mechanism that allows the tokens to "look back" and take help from all the previous tokens (exactly as you intuitively described!).
When he adds Self-Attention, this exact same generate() loop will stay exactly the same, but the model will suddenly start generating beautiful, coherent Shakespeare because the logits will actually contain information from the whole sequence!

How the for loop features actually working:

Step 1: we have make a loop, like the kalman filter, but we have to initialize it with some value like we did with matrix = 0 and then perform further operations and append it in the self loop

Similarly, first we are taking initial demo input as _('/n' space) and passing in the forward loop,
So the logits first value will be in dim of (B, T, C) size i.e. we will have raw outcomes of every alphabet, like how much probabililty is of that alphabet coming next after the input letter

And as it's the initial step, and no target is now pass or generated from the forward pass, loss = None

Step 2: so, now we have the (B, T, C) i.e. the possible outcomes (later probability), so we will take only the last input letter, alphabet outcome only, understand from below example

How PyTorch Slicing Works:

A 3D tensor has 3 slots: [dimension 0, dimension 1, dimension 2], corresponding to [B, T, C].

1. **: (first slot)**: Keep **all** batches ([Equation]).
2. **-1 (second slot)**: Pick **only the last token** in the sequence ([Equation]). Because we pick a single integer index rather than a slice, that dimension collapses and disappears!
3. **: (third slot)**: Keep **all** vocabulary scores ([Equation]).

Concrete Numerical Example:

Let's use small numbers so we can see every single value:

- Batch size [Equation]
- Sequence length [Equation] (e.g., words "c", "a", "t")
- Channels/Classes [Equation]

Our logits tensor of shape **(2, 3, 4)** looks like this:

Batch 0 ("c", "a", "t"):
Token 0 ("c"): [1.0, 0.5, -0.2, 2.1]
Token 1 ("a"): [0.1, 3.2, 1.1, -0.5]
Token 2 ("t"): [2.4, -1.0, 0.8, 4.5] <-- Last token (-1)

Batch 1 ("d", "o", "g"):
Token 0 ("d"): [0.0, 1.1, 2.2, -1.0]
Token 1 ("o"): [1.5, 0.2, -0.8, 1.9]
Token 2 ("g"): [3.1, 2.0, -0.4, 0.7] <-- Last token (-1)

Now we run:
last_logits = logits[:, -1, :]

→ IMP

PyTorch only grabs the rows marked **Last token (-1)**:

text
Row 0 (from Batch 0): [2.4, -1.0, 0.8, 4.5]
Row 1 (from Batch 1): [3.1, 2.0, -0.4, 0.7]
The sequence dimension ([Equation]) has been dropped! The new shape is **(2, 4)**, which is **(B, C)**.

Step 3: now after picking the last letter all the outcomes coming, we will convert them into probability → step 3

Using softmax function it's like a sigmoid function

And dimension refer to 3D tensor, it help to see how the each tensor row or we Can say each 2D tensor is stack on top of each other to make a 3D tensor, so that each 2 D tensor is represented as dim (dimension)

So, (B, T, C) -----> dim 0 dim 1 dim 2

Dim -1 means always the last dimension

What does dim=-1 tell Softmax to do?

dim tells Softmax: **"Across which direction should the numbers sum to 1.0 (100%)?"**
Because we set dim=-1, PyTorch applies Softmax **across the columns (the vocabulary classes)** for each row individually:

text
Raw Logits (B, C):
Batch 0: [2.4, -1.0, 0.8, 4.5]
Batch 1: [3.1, 2.0, -0.4, 0.7]

Apply Softmax along dim=-1 (across each row)
↓↓↓

Probabilities (B, C):
Batch 0: [0.10, 0.003, 0.02, 0.87] --> Sums to 1.0 (100%)!

Batch 1: [0.68, 0.23, 0.02, 0.07] --> Sums to 1.0 (100%)!

v.IMP

Why not dim=0?
If you accidentally set dim=0, PyTorch would normalize **downwards across the batches** (comparing Batch 0 with Batch 1), which makes no sense. We want the probabilities to sum to [Equation] across the **vocabulary choices** for each prediction!

Step 4: now, after we have the probabillity of each character, we need next index (idx_next), so that again when the loop continues from the new start, the next idex will be the new calculated index, like we do in simple loop
next_index = index

→ step: 4

But how do we choose the next character:
using the torch multinomial function

Torch.multinomial(probs, num_sample=1)

Example:

The Fundamental Difference

- **Greedy (argmax) is 100% Deterministic:**
  There is **zero chance or randomness**. If you run the code 1,000 times, you will get the **exact same text** every single time.
- **Sampling (multinomial) is Probabilistic:**
  There is **controlled randomness**. If you run the code 1,000 times, you get **different, unique variations** each time.

A Concrete Example

Imagine the model is predicting the word after: *"I would like a cup of..."*

- 'coffee' [Equation] probability

- 'tea' [Equation] probability

1. **With Greedy (argmax):**
- It looks at [Equation] .
- It **always** picks 'coffee'.
- It will **NEVER** pick 'tea'. In fact, 'tea' has a [Equation] **chance** of ever appearing, even though it was almost just as likely!

2. **With Sampling (torch.multinomial):**
- It respects the percentages.
- Out of 100 generations:
  - It will pick 'coffee' roughly **51 times**.
  - It will pick 'tea' roughly **49 times**.

And num_sample = 1 is the argument need to be passed with the multinomial function,
it gives which integer value i.e. the idx we will pick

Summarize from the code:

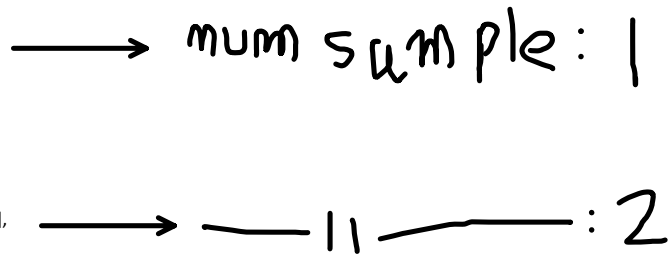
```
print(logits)
print(F.softmax(logits, dim=-1) )
print(torch.multinomial( F.softmax(logits, dim=-1) , num_sample=1)

Output:
tensor([[-1.5101, -0.0948,  1.0927,  ..., -0.6126, -0.6597,  0.7624],
        [ 0.3323, -0.0872, -0.7470,  ..., -0.6716, -0.9572, -0.9594],
        [ 0.2475, -0.6349, -1.2909,  ...,  1.3064, -0.2256, -1.8305],
        ...,
        [-2.1910, -0.7574,  1.9656,  ..., -0.3580,  0.8585, -0.6161],
        [ 0.5978, -0.0514, -0.0646,  ..., -1.4649, -2.0555,  1.8275],
        [-0.6787,  0.8662, -1.6433,  ...,  2.3671, -0.7775, -0.2586]],
       grad_fn=<ViewBackward0>)

tensor([[0.0021, 0.0085, 0.0280,  ..., 0.0051, 0.0048, 0.0201],
        [0.0095, 0.0063, 0.0032,  ..., 0.0035, 0.0026, 0.0026],
        [0.0118, 0.0049, 0.0025,  ..., 0.0341, 0.0074, 0.0015],
        ...,
        [0.0007, 0.0030, 0.0464,  ..., 0.0045, 0.0153, 0.0035],
        [0.0172, 0.0090, 0.0088,  ..., 0.0022, 0.0012, 0.0587],
        [0.0045, 0.0212, 0.0017,  ..., 0.0950, 0.0041, 0.0069]],
       grad_fn=<SoftmaxBackward0>)

tensor([[21],
        [24],
        [45],
        [22],
        [56],
        [ 7],
        [57],.....

tensor([[62, 58],
        [46, 10],
        [33, 58],
        [13, 38],
        [42, 64],
        [ 1, 12],
```



Step 5: the multinomial will give a index, from each row

Visual Example:

probs has shape **(B, C)** — let's say [Equation] batches and [Equation] characters:

```
text
                The 65 Character Probabilities (C)
Batch 0 (Row 0): [ 0.01,  0.70,  0.05, ... ]  --> Rolls die for Batch 0 --> Picks token 1  ('a')
Batch 1 (Row 1): [ 0.12,  0.03,  0.55, ... ]  --> Rolls die for Batch 1 --> Picks token 2  ('b')
Batch 2 (Row 2): [ 0.05,  0.10,  0.02, ... ]  --> Rolls die for Batch 2 --> Picks token 12 ('h')
Batch 3 (Row 3): [ 0.00,  0.80,  0.01, ... ]  --> Rolls die for Batch 3 --> Picks token 1  ('a')
```

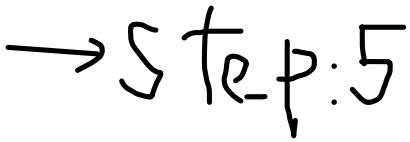
The Output idx_next of shape (B, 1):

```
text
[
  [ 1 ],   <-- Next character for Batch 0
  [ 2 ],   <-- Next character for Batch 1
  [ 12 ],  <-- Next character for Batch 2
  [ 1 ]    <-- Next character for Batch 3
]
```

Remember:
Here the batch are newly generated, as this are the batchs like:

Taking each integer number from each row, from that softmax tensor single row,
Hence here they are written as batch 1, batch 2, etc, and so on….

Step 5: at- last the token we have generate is append in a new list say idx and
After again then the loop start i.e. for 2 (second) iteration it will take the idx (of the first one, here, let's say _) and the idx next (the newly generate idx, let's say 'h') and the " new idx " will be pass in the step 1: logits, targets = self{idx) again, but this time combininhg both (_h)
And so on the loop continues….



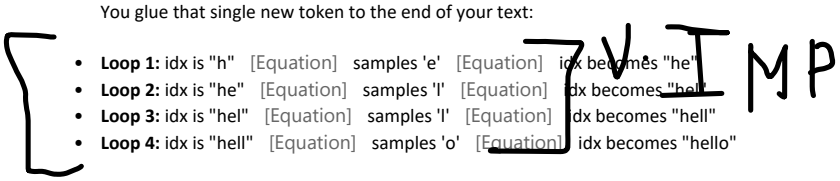


How it reassigns idx in the loop

Because num_samples = 1, idx_next has shape **(B, 1)** (one token ID per batch).
Then, on the very next line:

python
```
idx = torch.cat((idx, idx_next), dim=1)
```

You glue that single new token to the end of your text:



- **Loop 1:** idx is "h" [Equation] samples 'e' [Equation] idx becomes "he"
- **Loop 2:** idx is "he" [Equation] samples 'l' [Equation] idx becomes "hel"
- **Loop 3:** idx is "hel" [Equation] samples 'l' [Equation] idx becomes "hell"
- **Loop 4:** idx is "hell" [Equation] samples 'o' [Equation] idx becomes "hello"

Finally we will write:

```
M = BigramLangaugeModel(vocab_size)
Idx = tensor.zeros((1, 1), dtype=torch.long) )
Generate_tokens = m.generate(idx, max_new_token=100)
Print(decode(generate_token[0].tolist() ) }
```

Output:
```
EFSYZrRzlOwtmxrHF-T 3Uk$sMQal!bOA;ary;bd'qd!mUHeDudrbUH'3q
aA;bFvJH'mgVNo3hsxMQ,VcjL yNMDwan-'crmumS
```

As we have not train the model yet, it gives the gribbish value

1. Why [0]? (Removes the Batch dimension)

When m.generate() finishes, generate_token is a **2D PyTorch Tensor** of shape **(B, T)**:

- Batch size [Equation]
- Total tokens [Equation] (1 seed token + 100 generated tokens)

It looks like this:

python
```
generate_token = tensor([[ 0, 34, 12, 45, ... ]])  # Notice the DOUBLE brackets [[ ]]
```

```
generate_token = tensor([[ 0, 34, 12, 45, ... ]])  # Notice the DOUBLE brackets [[ ]]
```

When you do generate_token[0]:

- You grab the **first sequence** from the batch.
- It drops the batch dimension, turning it into a **1D Tensor** with single brackets:

python
```
generate_token[0] = tensor([ 0, 34, 12, 45, ... ]) # Single brackets [ ]
```

2. Why .tolist()? (Tensor [Equation] Standard Python List)

generate_token[0] is still a **PyTorch Tensor object** (torch.Tensor).
However, your decode function was written using Python dictionary lookups:

python
```
decode = lambda l: ''.join([itos[i] for i in l])
```
The decode function expects a plain Python list of integers like [0, 34, 12, ...].
Calling **.tolist()** converts the PyTorch Tensor into a native Python list:

python
```
# Before .tolist():
tensor([0, 34, 12, 45])   # PyTorch Tensor object
# After .tolist():
[0, 34, 12, 45]           # Plain Python list of integers
```

Quick Summary

Step	Code	What it does	Type / Shape
Output of model	generate_token	Raw model output	2D Tensor (1, 101)
Select 1st batch	generate_token[0]	Extracts the sequence	1D Tensor (101)
Convert to list	generate_token[0].tolist()	Converts tensor to Python list	Python list
Decode to text	decode(...)	Maps integers to letters via itos	Python str (text)